PROGRAMAÇÃO E ARQUITETURA DE COMPUTADORES PARALELOS

Trabalho Prático 01

Este trabalho tem como objetivo desenvolver o seu entendimento sobre paralelismo de CPU, expondo de forma prática os conceitos de SIMD, multicore e hyperthreading. Você deverá escrever um relatório sobre experimentos realizados em três aplicações diferentes, focando na explicação dos resultados e como eles se comparam aos desempenhos teóricos máximos.

Para isso, estão disponíveis três pequenas aplicações em C/C++ usando a biblioteca ISPC para execução paralela. Você deverá trabalhar modificando esses códigos para realizar seus experimentos. A tarefa de programação não é extensa, então não será necessário submeter códigos. Porém, isso não significa que são avaliações fáceis de serem feitas. Como o objetivo deste trabalho é explorar os tipos de paralelismos disponíveis para CPUs a baixo nível, é esperado que os desempenhos encontrados nos experimentos sejam analisados em comparação ao desempenho teórico máximo esperado. Esse valor máximo também deve ser justificado tendo em vista a CPU usada para conduzir os experimentos.

Ambiente de Execução

É recomendada a realização dos experimentos em ambiente Linux com hardware x86 por ser mais previsível e por ter uma documentação mais disponível. Isso facilitará o entendimento a baixo nível do que é feito pelo SO e pela CPU. Experimentação em outros SOs pode ser realizada, mas a seu risco. Já o uso de hardware que não seja x86 (e.g., alguns Macbooks) pode complicar o entendimento da capacidade de paralelismo da CPU. Como o objetivo é o relatório, não há problemas desde que os resultados sejam válidos e explicados corretamente.

Para usar o ISPC basta apenas baixar o binário pré-compilado e adicioná-lo ao PATH com os comandos:

```
wget https://github.com/ispc/ispc/releases/download/v1.24.0/ispc-v1.24.0-linux.tar.gz
tar -xvf ispc-v1.24.0-linux.tar.gz
export PATH=$PATH:${PATH_DO_DOWNLOAD}/ispc-v1.24.0-linux/bin
```

Todos os códigos foram testados para compilar nativamente com um make e com a versão acima do ISPC.

Relatório e Entregas

O relatório a ser submetido deve ter no máximo **5 páginas**, podendo ter figuras, gráficos ou tabelas. Caso for usar figuras, avalie se é realmente necessário para explicar seu ponto, pois o espaço é escasso. Uma recomendação de formato simples é: (1) avaliação da máquina usada/especificação para executar os experimentos, (2-4) os três experimentos, e (5) análise final. Em seguida são apresentados os experimentos a serem executados e avaliados.

Este trabalho pode ser feito em **grupo de 2 pessoas**. Apenas um dos membros deve submeter o relatório final. A data final de entrega é **21/09 às 23:50**. Atentem-se ao prazo, pois... perde ponto por dia ...

Experimento 1: Mandelbrot

Em tp1/exp1 está o código para a geração de uma imagem fractal para o Conjunto de Mandelbrot. Este é o momento para se familiarizar com a execução de código paralelo com ISPC. Para compilar o código basta um make. O executável de saída (mandelbrot_ispc) não precisa de nenhum parâmetro de entrada, gerando as imagens de saída e informando o tempo de execução de cada versão e o speedup comparado à versão serializada. A versão ISCP usa apenas 1 core e instruções AVX2-x8 (definidas no Makefile). Isso significa que existe execução vetorizada com 8 elementos processados ao mesmo tempo via SIMD (quando possível). Também é possível executar a versão vetorizada em várias threads (i.e., multicore) usando o argumento -t em mandelbrot_ispc. Por fim, existem duas gerações de imagens diferentes que podem ser feitas, usando o argumento -v2 (abram as imagens de saída para ver o que acontece).

O que deve ser feito e discutido:

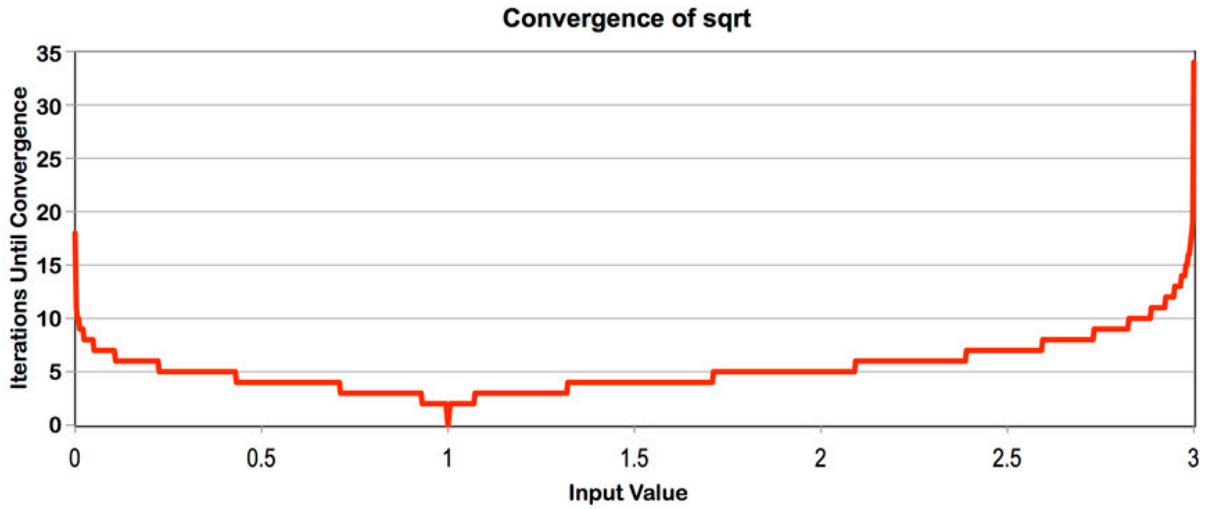
1. Compile e rode o código sem threads (apenas com as versões serializada e SIMD). Qual foi o speedup retornado? Dada a CPU usada, qual seria o speedup máximo que poderia aparecer? Como o speedup real se compara com o máximo teórico? Existe alguma diferença significativa? Se sim, por que?
2. Rode as duas versões de imagens. Existe diferença nos speedups do ISPC vs serializado para as duas imagens? Se sim, por que uma imagem roda melhor/pior no ISCP?
3. Rode o código com threads. O que acontece quando você altera o número de threads submetidas para execução? Qual foi a melhor configuração encontrada para a sua máquina? Como esse valor se relaciona com o número de cores/threads da sua CPU?

Experimento 2: sqrt

Em tp1/exp2 está o código para executar um algoritmo de aproximação de raiz quadrada (sqrt). Nele é gerado um array com 20 milhões de valores aleatórios, no qual executamos a aproximação de sqrt valor a valor. São disponibilizadas 3 implementações: (i) serializada, (ii) com SIMD (vec) e (iii) com SIMD em várias threads (thd). Assim como anteriormente, a saída mostra os tempos de execução e os speedups comparados à versão serializada.

O que deve ser feito e discutido:

1. Compile e rode o executável. Como os speedups se comparam aos máximos teóricos esperados para a sua CPU?
2. O algoritmo de aproximação de sqrt não tem tempo de execução constante para qualquer valor de entrada. Na figura abaixo temos o número de iterações necessárias para se chegar à convergência para diferentes valores de entrada. Mantendo o número de 20 milhões de valores de entrada, construa um array de entrada diferente que **maximize** o speedup da versão vec comparada à versão serializada.
3. Agora crie um array de entrada de 20 milhões de valores que **minimize** o speedup da versão vec comparada à versão serializada.
4. Não se esqueça de explicar por que os seus arrays minimizam ou maximizam o speedup de vec. Compare esses novos speedups com o máximo teórico da sua CPU.



Experimento 3: BLAS com saxpy

Uma operação fundamental para bibliotecas de BLAS (Basic Linear Algebra Subproblems) é a saxpy (single precision AX + Y). Em tp1/exp3 está o código para executar saxpy em dois arrays de 20 milhões de valores cada, com um escalar fixo. Novamente, são disponibilizadas as três implementações: (i) serializada, (ii) com SIMD (vec) e (iii) com SIMD em várias threads (thd).

O que deve ser feito e discutido:

1. Compile e rode o código. Compare as versões serializadas e vec, avaliando quão próximo o speedup de vec está do máximo teórico.
2. Compare a versão vec com a thd, avaliando quão próximo o speedup de thd está do máximo teórico.
3. Tem como melhorar o desempenho inicial de thd? Que alterações no código (sem alterar os inputs) você poderia fazer para melhorar o desempenho de thd? Por que isso funcionaria/funcionou?

Análise final: Comparando os workloads

Os códigos executados foram implementados e otimizados de forma relativamente parecida, porém você deve ter encontrado divergências nos speedups entre os diferentes workloads. Explique agora como os diferentes workloads podem influenciar o desempenho de algoritmos e execuções paralelas com SIMD e multicore/multithread.

Dicas e observações

- Um gráfico de interesse para plotar é o de strong-scaling, onde, dada uma entrada fixa, o número de recursos (nós, cores, threads, ...) é variado.
- Avaliem bem se uma figura ou gráfico é necessário já que o espaço é escasso.
- Nos makefiles existe uma configuração do target de SIMD. Ou seja: tem como configurar para mandar instruções SIMD de 4 valores em vez de 8 valores (default). Rodar SIMD com uma largura menor pode te ajudar a ver o padrão de desempenho e eficiência do uso de SIMD.
- Não troquem o SIMD de avx2 para avx512. Dependendo da CPU, a implementação do avx512 pode ser bem diferente, resultando em tempos estranhos e difíceis de entender.
- Para aqueles já com experiência, ou curiosos, Latex é legal para montar esses relatórios.